

CS 421 Examination

Functional Programming Paradigms – Answer Key

1. **(A)** It always returns the same output for the same input and has no side effects.
Pure functions are deterministic and have no observable effects beyond computing their return value. They don't modify external state, perform I/O, or depend on mutable data. This makes them easy to test, reason about, and parallelize.
2. **(B)** Only when their values are needed.
Lazy evaluation (or call-by-need) delays computation until the result is required. This enables working with infinite data structures and can improve performance by avoiding unnecessary computations. Haskell uses lazy evaluation by default.
3. **(B)** A type class that provides a way to chain operations while abstracting computational context.
A monad encapsulates computations with context (like optional values, state, I/O). It provides `bind` (or `>>=`) to chain operations and `return` to inject values. Examples: `Maybe`, `List`, `IO` monads.
4. **(A)** Functions can be replaced by their return values without changing program behavior.
Referential transparency means an expression always evaluates to the same value and can be replaced by that value anywhere. This is a consequence of purity and enables equational reasoning: you can substitute equals for equals, like in algebra.

5. *Sample answer:*

Higher-order functions: Functions that take other functions as arguments or return functions as results.

Benefits for reusability:

- Abstract common patterns, parameterized by behavior
- Reduce code duplication by separating traversal logic from business logic
- Enable declarative, composable code

Examples:

- **map:** Applies a function to each element: `map(f, [1,2,3]) → [f(1),f(2),f(3)]`.
Reuses iteration logic for any transformation
- **filter:** Selects elements matching a predicate: `filter(isEven, [1,2,3,4]) → [2,4]`.
Reuses selection logic
- **reduce (fold):** Combines elements using a binary operation: `reduce(+, [1,2,3]) → 6`.
Generalizes aggregation

These higher-order functions eliminate explicit loops and promote composition.

6. *Sample answer:*

Eager evaluation: Expressions evaluated immediately when bound to variables (default in most languages).

Lazy evaluation: Expressions evaluated only when needed (default in Haskell).

Advantages of lazy evaluation:

- Avoids unnecessary computation (short-circuiting naturally)
- Enables infinite data structures
- Improves modularity: separate data generation from consumption

Pitfalls:

- Harder to reason about time and space complexity
- Risk of space leaks (accumulating thunks)
- Debugging is more complex

Example - Infinite list:

```
-- Generate infinite list of natural numbers
nats = [0..]

-- Take first 5 elements
take 5 nats -- [0,1,2,3,4]
```

Only the needed elements are computed; the rest remain unevaluated thunks.

7. *Sample answer:*

Currying: Transforming a function with multiple arguments into a sequence of functions, each taking a single argument.

```
-- Uncurried: f(x, y) = x + y
-- Curried: f(x)(y) = x + y

add :: Int -> Int -> Int
add x y = x + y

-- add is automatically curried in Haskell
addFive = add 5 -- partially applied
addFive 3 -- returns 8
```

Partial application: Fixing some arguments of a function to create a new function with fewer parameters.

Difference: Currying is a transformation of function structure; partial application is using that structure to create specialized functions.

Benefits:

- Create specialized functions from general ones
- Improve function composition
- Enable point-free style (tacit programming)
- Example: `map (+ 1) [1,2,3]` uses partial application of `(+)`

8. *Sample answer:*

Immutability: Data cannot be modified after creation. Instead of changing values, create new values.

Importance:

- **Easier reasoning:** No hidden state changes; functions depend only on inputs
- **Thread safety:** Immutable data is inherently thread-safe (no race conditions)
- **Time-travel debugging:** Can save and restore program state
- **Caching:** Safe to cache function results (pure functions with immutable inputs)

Concurrency benefits: In concurrent programs, mutable shared state requires locks, leading to deadlocks and race conditions. Immutable data eliminates these issues—threads can safely share references without synchronization.

Performance trade-offs:

- **Cost:** Creating new objects instead of mutating can be expensive
- **Mitigation:** Persistent data structures (structural sharing) reuse parts of old structures, reducing copying overhead. Example: persistent trees share unchanged branches
- Modern FP languages optimize heavily for immutability